



OPEN

DATA DESCRIPTOR

F-DATA: A Fugaku Workload Dataset for Job-centric Predictive Modelling in HPC Systems

Francesco Antici¹✉, Andrea Bartolini¹, Jens Domke², Zeynep Kiziltan³ & Keiji Yamamoto²

In the last decades, High Performance Computing (HPC) systems have accelerated scientific discoveries and innovations across different domains, from epidemic studies to climate science. For sustainable development of HPC systems, it is fundamental to address their environmental impact regarding carbon footprint emission and energy requirement, while ensuring high system throughput. Analyzing and predicting HPC job execution characteristics is instrumental in developing workload management strategies to simultaneously optimize the system throughput and minimize the environmental impact. However, model development for accurate predictions is hindered by lack of voluminous public datasets. In this paper, we present F-DATA, a public dataset containing the information of around 24 million jobs executed on Fugaku, the most powerful supercomputer during the data collection phase. The data contains an extensive set of features, allowing for a multitude of job characteristics prediction. The sensitive job data appears both in anonymized and irreversibly encoded versions. The encoding is based on a Natural Language Processing model and retains sensitive but useful job information for prediction purposes without violating privacy concerns.

Background & Summary

High-performance computing (HPC) systems are sophisticated machines, which leverage distributed hardware architectures and parallel processing techniques to solve complex problems that are beyond the reach of conventional computers. These systems have greatly accelerated recent scientific discoveries and innovations across different domains, from weather forecasting¹ and drug discovery² to epidemic studies^{3,4} and environmental science⁵. Given their central role in modern society, it is fundamental to ensure their sustainable development, deployment, and usage. This implies addressing environmental concerns regarding carbon footprint emissions and energy requirements, while ensuring optimal system throughput (i.e. the amount of tasks processed by the system in a certain amount of time). An HPC job is a computational task (i.e. an application together with its input), submitted by a user and executed on system resources, such as CPU, GPU and RAM. As shown in past work^{6–8}, predicting job execution characteristics can be instrumental to develop workload management strategies aiming at simultaneously optimize the system throughput and environmental impact.

Job execution characteristics can be effectively predicted with Machine Learning (ML) models trained on voluminous datasets, as shown in many past works^{9–11}. However, public datasets are currently scarce, and they do not contain rich information about jobs; this is due to two main reasons. First, privacy concerns prevent the disclosure of some sensitive job data^{12,13}, such as *user name* and *job name*. Second, due to technical difficulties in the data collection (e.g. deployment of specific software which continuously monitors and profiles all the job executions^{14–16}), public resources do not contain some important job execution characteristics, such as *power consumption* and *performance metrics* (e.g. *number of floating-point operations per second*, *memory bandwidth* and *memory/compute-bound class*).

Prior work^{9,11} shows that *user name* and *job name* reveal insights on users' job execution patterns, and they are crucial to develop accurate predictive models. Conversely, information such as *power consumption* and *performance metrics* can help design workload management strategies to improve system throughput and energy efficiency^{8,17,18}. Unfortunately, some existing datasets, such as¹⁶, contain the *power consumption* of only a fraction of the jobs executed on the system, which hinders the development of system-level strategies. Instead, no public

¹University of Bologna, Department of Electrical, Electronic, and Information Engineering Guglielmo Marconi, Bologna, 40121, Italy. ²RIKEN Center for Computational Science, Kobe, Japan. ³University of Bologna, Department of Computer Science and Engineering, Bologna, 40121, Italy. ✉e-mail: francesco.antici@unibo.it

resource contains high-quality *performance metrics* suitable for understanding and classifying jobs computational characteristics, which are essential to ML training for sustainable computing, and analysis of the system's efficiency and usage. Hence, previous attempts at developing sophisticated scheduling and run-time techniques, as well as analysis of the system's computational efficiency, were narrow in scope and fell short with respect to the expected throughput enhancements.

To address all the aforementioned problems, we present F-DATA, a workload dataset for job-centric predictive modeling in HPC systems, and for system efficiency and usage analysis. The dataset collects data of job executions on the Supercomputer Fugaku (<https://www.fujitsu.com/global/about/innovation/fugaku/>), during more than three years of system usage. Fugaku is a production HPC system hosted at the RIKEN Center for Computational Science, in Japan (<https://www.riken.jp/en/research/labs/r-ccs/>). It was deployed in production in 2020, and more than 4 years later it is still in the top positions of two important lists of the world's most powerful supercomputers: 1st in the HPCG list¹⁹ and 6th in the TOP500 list²⁰ (as of November 2024). The system was developed by RIKEN and Fujitsu, and it can reach a peak performance of 537 PFlops/s through around 160k interconnected computational nodes, each of them endowed with 48 Arm cores and 32 GiB of high-bandwidth memory (HBM). The dataset is publicly available in Zenodo²¹, and it contains extensive information on around 24 million job executions. The sensitive job data appears both in anonymized and irreversibly encoded versions. The encoding is based on a Natural Language Processing (NLP) model and as confirmed by our experimental study, it retains sensitive but useful job information for prediction purposes without violating the system users' privacy.

F-DATA is the first public dataset containing i) the data of one of the most powerful supercomputers in the world, ii) various job features (such as *performance metrics*, *power consumption*, *duration*, and *exit state* among others), which are useful for a multitude of analysis and job-centric predictive modelling, and iii) an irreversible encoding of the sensitive data. By releasing F-DATA publicly, we empower HPC researchers and practitioners with a resource to foster the development of ML-based predictive models aimed at guaranteeing the sustainable development of HPC systems.

Methods

In this section, we discuss the steps and methods used to extract and create the dataset. We describe the original job features, how we derive new features from the original ones, and how we protect and encode the private information.

Data Extraction. The data extraction is achieved through a proprietary operations management software installed on Fugaku²², which enables the recording and storage of job data in an instance of a PostgreSQL database²³. We query the database via its interface, and we retrieve the data of the jobs executed on the system between March 2021 and April 2024. We consider the data as of March 2021, when the system left the pre-production phase and became available for general usage.

We note that the raw data used to generate this dataset are property of RIKEN. Hence, their distribution is subject to RIKEN's discretion and regulation. Access to the raw data can be granted upon agreement between RIKEN and the requesting party. In case of interest in obtaining the raw data, we are available to help and facilitate the connection with RIKEN representatives.

Original Job Features. Each job data extracted from the database contains features concerning the job submission, execution and completion. The first category includes the information available at job submission time, such as the job's *user information* (*usr* in Table 2), the *submission time* (i.e. when the user submits the job to the system, *adt* in Table 2) and the *requested resources* by the user (e.g. *number of cores*, *amount of memory*, *number of nodes*, *node frequency*, appearing as *cnumr*, *mszl*, *nnumr* and *freq_req* in Table 2, respectively). When the job starts running, the execution features can be collected, such as the *start time* (i.e. when the job starts, *sdt* in Table 2) and the actual amount of *resources allocated* to the job (e.g. *cnumat*, *msza*, *nnuma* and *freq_alloc* in Table 2).

Job execution characteristics. At job completion, it is possible to access execution outcome characteristics, such as the *resources used* (e.g. *cnumut*, *mmszu*, and *nnumu* in Table 2), the *duration*, the *exit code*, the *power consumption* and the *performance counters*. The *exit code* (*ec* in Table 2) is an integer value in the range [0-255] representing whether the job execution was successful or not. The *power consumption* is the sum of the minimum, average, or maximum power consumption of the resources allocated to the job during its execution (*minpcon*, *avgpcon*, *maxpcon* in Table 2), and it can be collected from different hardware components, such as mainboard, CPU and RAM. The *performance counters* (*perf1-perf6* in Table 2) store the amount of hardware-related operations (e.g. number of memory read/write requests and floating-point operations) performed by the job, which allows us to gain insights on the job's resource utilization. The job data includes also features on how the job utilizes the resources allocated. Specifically, the *average idle time* (*idle_time_ave* in Table 2) stores the amount of time the job was idle (i.e., not performing any operations on the resources); conversely, the *cpu time* (*uctmut*, *sctmut* and *usctmut* in Table 2) reports the total time the job has performed operations on the CPU. Such features are collected by the Fugaku operational manager software. This software employs a low-level proprietary profiler which automatically monitors the job executions, and saves a series of aggregated metrics after their completion.

Job scheduling features. The Fugaku operational manager software also collects information concerning the workload manager software, including the job scheduler. While the characteristics of the internal scheduling algorithm are not disclosed publicly, the database includes per-job information, which allows for the reproduction of the whole job scheduling process, such as the *submission time* (*adt* in Table 2), *scheduling time* (*schedsdt* in Table 2), *queue time* (*qdt* in Table 2), *start time* (*sdt* in Table 2) and *end time* (*edt* in Table 2). Features like

File name	# of jobs	Size (in MB)
21_03.parquet	713,617	458
21_04.parquet	116,977	150
21_05.parquet	595,314	433
21_06.parquet	584,505	505
21_07.parquet	338,251	282
21_08.parquet	549,070	336
21_09.parquet	402,341	331
21_10.parquet	641,820	838
21_11.parquet	883,186	699
21_12.parquet	931,118	931
22_01.parquet	1,307,407	1,300
22_02.parquet	1,018,106	817
22_03.parquet	678,156	746
22_04.parquet	424,413	629
22_05.parquet	422,400	536
22_06.parquet	473,224	586
22_07.parquet	460,411	575
22_08.parquet	729,651	634
22_09.parquet	660,134	841
22_10.parquet	533,978	663
22_11.parquet	814,384	795
22_12.parquet	794,880	855
23_01.parquet	894,491	981
23_02.parquet	636,416	762
23_03.parquet	684,593	728
23_04.parquet	618,335	575
23_05.parquet	610,437	635
23_06.parquet	1,303,631	1,300
23_07.parquet	961,051	968
23_08.parquet	1,043,535	1,200
23_09.parquet	572,207	455
23_10.parquet	808,424	885
23_11.parquet	730,023	944
23_12.parquet	626,404	909
24_01.parquet	705,516	1,100
24_02.parquet	669,758	800
24_03.parquet	508,286	529
24_04.parquet	420,450	568

Table 1. # of jobs and size (in MB) of all the F-DATA parquet file chunks.

the the *scheduling time* and the *queue time* provide insights on the internal scheduling process. Specifically, the former is the timestamp of when the scheduling decision is obtained. This operation establishes when and on which resources to execute the job. The latter is the timestamp of when the job enters the execution queue, after the scheduling decision is performed. By considering these features in conjunction with the *submission time* and *start time* of the job, it is possible to derive useful metrics, such as the time needed to perform the scheduling decision, and the time the job awaits in the queue before execution. Such information is instrumental to analyse how different job characteristics can impact the scheduling decisions and the time the job requires to be processed.

We extend the job data by deriving new job features from the original features, and by encoding the sensitive data, which we explain hereafter. The full list of the 45 job features and their description are reported in Table 2.

Derived Features. For each job, we derive the *exit state* and a series of *performance metrics* features, starting from the *exit code* and *performance counters* features. The *exit state* of a job is a label that describes the outcome of the execution, which can be successful or not. As explained in the literature¹¹, this feature is directly related to the job *exit code* which is 0 if a computation ends without any error, or an integer number in the range [1-255] in case of errors. Hence, we label the *exit state* of a job as *completed* if the *exit code* is 0, and as *failed* otherwise. We cannot preclude that a user's application intentionally returns a non-0 exit code despite running to completion, and hence the number of presumably failed jobs, shown hereafter, may include false-positives.

Column	Description	Type	Anonymized
jid	The id assigned to the job when it is submitted	string	✓
usr	The username of the user submitting the job	string	✓
jnam	The name of the job	string	✓
cnumr	Number of cores requested for the job	int	X
cnumat	Number of cores allocated to the job	int	X
cnumut	Number of cores used by the job	int	X
nnumr	Number of nodes requested for the job	int	X
adt	Arrival datetime (submission time)	datetime	X
qdt	Datetime of insertion in the job queue after the scheduling choice is performed	datetime	X
schedsdt	Datetime of performed scheduling choice after the job submission	datetime	X
deldt	Datetime of job deletion in case the job is deleted by the user	datetime	X
ec	Job exit code	int	X
elpl	Elapsed time limit (the time limit set to the job duration)	float	X
sdt	Start datetime	datetime	X
edt	End datetime	datetime	X
nnuma	Number of nodes allocated to the job	int	X
idle_time_ave	The average time spent by the job not performing any operations on the CPUs	float	X
nnumu	Number of nodes used by the job	int	X
perf1	Number of execution cycles	float	X
perf2	Number of Advanced SIMD arithmetic operations (FP_FIXED_OPS_SPEC event in ⁵³)	float	X
perf3	Number of SVE arithmetic operations (FP_SCALE_OPS_SPEC event in ⁵³)	float	X
perf4	Amount of read transactions of the memory (BUS_READ_TOTAL_MEM event in ⁵³)	float	X
perf5	Amount of write transactions of the memory (BUS_WRITE_TOTAL_MEM event in ⁵³)	float	X
perf6	Number of sleep cycles	float	X
pri	Priority	int	X
econ	Energy consumption	float	X
avgpcon	Sum of the average power consumption of the nodes allocated to the job	float	X
minpcon	Sum of the minimum power consumption of the nodes allocated to the job	float	X
maxpcon	Sum of the maximum power consumption of the nodes allocated to the job	float	X
mszl	Limit of the amount of memory allocated to the job (requested by the user)	float	X
msza	Amount of memory allocated to the job	float	X
mmszu	Amount of memory used by the job	float	X
uctmut	User CPU time total use	float	X
sctmut	System CPU time total use	float	X
usctmut	Total CPU time use	float	X
jobenv_req	Job environment requested	string	✓
freq_req	Node frequency requested	int	X
freq_alloc	Node frequency allocated	int	X
flops	Number of floating point operations per second	float	X
mbwidth	Memory bandwidth	float	X
opint	Operational intensity	float	X
pclass	Performance class (either compute-bound or memory-bound)	string	X
embedding	Sensitive data encoding with SBert	Array[float]	X
exit state	Exit state of the job execution (either completed or failed)	string	X
duration	Duration of the job execution in seconds	float	X

Table 2. List of all the columns (job features) of F-DATA, along with their description, data type and whether they are anonymized or not.

The *performance metrics* provide high-level information on the job resource utilization. Such information is fundamental to characterize a job execution, aiming to improve both job and system level throughput and energy efficiency^{8,17,24}. The job *performance metrics* we compute are *#flops*, *mbwidth*, *opint* and *pclass* and rely on *performance counters* (*perf2*–*perf6*). The *#flops_j* is the number of floating-point operations per second performed by the job *j* and is computed via Equation (1). In this equation, *perf2_j* is the fixed amount of operations, while *perf3_j* is the number of operations per CPU vector register, here 128-bit, which is then multiplied by 4 since Fugaku's A64FX CPU employs 512-bit long scalable vector register (so called SVE instructions). The memory bandwidth *mbwidth_j* is the amount of memory bytes moved per second during execution. In Equation (2), *perf4_j* and *perf5_j* are summed in order to obtain the total number of requests to the memory, as they represent the amount of memory read and write requests, respectively. Then, they are multiplied by the size of the memory

requests (i.e. 256 bytes of cache line size) to obtain the total amount of memory bytes moved. The compute cores of Fugaku are grouped by 12, forming the so-called Core Memory Groups (CMGs), since each group of 12 cores shares the same Level 2 cache and high-bandwidth memory (HBM) stack. Due to the fact that the $perf4_j$ and $perf5_j$ values are generated by summing all the values collected by each core for the whole CMG, these values need to be divided by 12 to eliminate redundant information. Since both $\#flops$ and $mbwidth$ are computed per second, we divide the values by the job duration ($duration_j$). The operational intensity $opint$, which is the amount of floating point operation per byte of the job execution, is computed as the ratio between $\#flops$ and $mbdwidth$.

$$\#flops_j = \frac{perf2_j + (perf3_j * 4)}{duration_j} \quad (1)$$

$$mbwidth_j = \frac{(perf4_j + perf5_j) * 256}{duration_j * 12} \quad (2)$$

Finally, we create the performance class label $pclass$, which can be either *memory-bound* or *compute-bound*. They refer to the jobs whose performance is bound by the memory access rate or by the system's arithmetical performance, respectively. We generate this job feature as shown in past work²⁵, by computing the *ridge point* of the system²⁶, which represents the ratio between the system's highest attainable performance (maximum number of floating-point operations per second) and memory bandwidth. We label all the job executions with $opint$ greater than *ridge point* as *compute-bound*, and the others as *memory-bound*.

Sensitive Data Anonymization. Publication of job data is possible upon effective protection of the sensitive data of the users and the system^{13,27}. Anonymization²⁸ is one of the most used techniques to protect sensitive data, and it consists of altering data in a way that prevents the original information to be identified. In F-DATA, the feature values requiring anonymization are *user information* (usr in Table 2), *job name* ($jnam$ in Table 2), *job id* (jid in Table 2) and *job environment* ($jobenv_req$ in Table 2). Those values could indeed reveal the user identity, thus violating personal privacy, and disclose confidential details about the research or work being conducted, which could violate internal privacy policies on intellectual property or non-disclosure-agreements. For analysis purposes, such features are kept in the dataset, but they are transformed as follows. For each feature, we take the list of all the values, without duplicates. As the data are originally ordered chronologically, the values list will be ordered by the time of first appearance in the dataset. The list index i is then used to generate the anonymization for a value of a feature f , as f_i (e.g. the first usr in the dataset becomes usr_0).

We note that public scientific computing clusters in general allow each user to see the information of other user's submitted jobs, meaning that a malicious actor could easily get Fugaku access via the HPCI trial access accounts and simply monitor the batch queue to gain access to sensitive information (such as *user information* and *job name*) and timings. Therefore, we believe that our approach is an appropriate anonymization strategy for our purposes. Besides, the adopted procedure has been internally approved before releasing the dataset.

Understanding which entities use the system and how they do so is crucial for ensuring accountability regarding HPC energy consumption and its environmental impact. Anonymizing sensitive data does not compromise transparency, as system users are required to agree to periodic public reporting of supercomputer usage statistics—often mandated by the funding agency—thereby enabling accountability. In fact, the agency responsible for managing system allocation (HPCI) publishes periodic reports (https://www.hpci-office.jp/en/achievements/user_report) detailing the workloads and projects executed on the systems. These reports include information on which scientific areas consume node-hours and the number of compute cycles processed. Accessing these public reports enables further analysis of the projects and workloads executed on the system. For example, it allows for examining shifts in workload composition following the release of ChatGPT, identifying the most frequently run applications, or evaluating which scientific fields have the largest environmental impact.

Sensitive Data Encoding. Using anonymized data may compromise the effectiveness of prediction models¹³. In the context of job-centric ML-based predictive modeling, previous work^{9,11} showed that encoding job data with an NLP model improves the prediction performance of ML models, with respect to using the data in the standard integer format. Thus, we encode the de-anonymized version of the sensitive data with an NLP model and add it to the dataset as the *sensitive data encoding* feature (*embedding* in Table 2).

Following the approach of related work^{9,11}, we rely on the NLP model SBert²⁹, a state-of-the-art sentence embedding model. SBert is obtained by fine-tuning pre-trained BERT (Bidirectional Encoder Representations from Transformers)³⁰ models on sentence similarity tasks. The model is built to understand the content of sentences or pieces of text and encode them with semantically meaningful sentence embeddings. The resulting representation of a text string from SBert is a fixed-size 384-dimensional floating-point array, which retains the semantic meaning of the original data without disclosing its content. We implement SBert leveraging the *sentence transformers* library (<https://www.sbert.net>), with the pre-trained model *all-MiniLM-L6-v2*, since it has the best trade-off between prediction quality and speed²⁹.

The *sensitive data encoding* feature for a job data is generated by merging the de-anonymized *user information*, *job name* and *job environment* features into a comma-separated string, and then encoding it with SBert. To this end, we do not consider the *job id* for the encoding, as it is an integer number and its original format does not provide any further information on the job nature with respect to the anonymized one. It is not possible to

recreate the original values from the *sensitive data encoding*³¹. We thus safely include it in the dataset, aiming to foster the development of effective predictive models, without violating the users' privacy.

Data Records

F-DATA is hosted in Zenodo²¹ and composed of 38 smaller chunks (for a total of 28 GB of data), named as YY-MM, each containing the data of the jobs executed during the month (MM) of the year (YY). Table 1 lists all the 38 files, along with the amount of jobs contained and the size in MB. We divide the files to make the dataset easier to download and access, and allow for experiments on smaller subsets of the stored information.

The files are saved in the *parquet* format, which leverages efficient compression and encoding schemes to create files of smaller sizes. All the files contain dataframes with the same set of columns (referring to job features), listed and described in Table 2 (where the feature names are written as they appear in the dataset, while in the paper text we rename them to ease readability). The information included in the dataset allows for a multitude of job-centric prediction tasks. For instance, it allows for the prediction of job and system level power consumption^{9,32,33}. By providing *minpcon*, *avgpcon* and *maxpcon*, jobs can be characterized in terms of full power consumption profile. Besides power, it is also possible to explore the energy consumption prediction task, as each job data contains the *econ* feature giving the energy consumption in Watt-Hours. Such values can be used to estimate the carbon footprint of a job execution, which is fundamental to guide the sustainable deployment of HPC systems³⁴. Moreover, all the *performance metrics* can be used as a target for a prediction model. Features like the *mbwidth*, *#flops* and *pclass* can be predicted to develop both co-scheduling techniques and specific hardware-software co-design techniques to improve system throughput and energy consumption, as demonstrated in the literature^{8,17,24,35,36}. Furthermore, researchers have shown how to use the *exit state* for job failure prediction tasks^{11,37}. Such an information might be useful to develop failure-aware scheduling strategies or checkpoint/restart libraries to minimize the system resource wastage³⁸. Finally, it is possible also to predict values related to the execution time of the job, such as *duration*¹⁰ or *end time*³⁹. These predictions can be used to develop better scheduling strategies which would be accounting for more realistic job durations⁴⁰, since users tend to overestimate the required runtime limits to avoid premature or accidental terminations of their calculations.

Job Data Analysis. In order to demonstrate the characteristics of the job data contained in F-DATA, in Figs. 1–6 we show and comment on the distribution of several job features. More specifically, in Fig. 1, we show the distribution of the amount of job executions divided by month. We observe no clear patterns or seasonality. Except for April 2021, the amount of data is steadily over 300K jobs a month, with peaks in June 2023 (more than 1.2 million of data) and in January 2022 (around 1 million of data).

Jobs in HPC systems are executed on a set of nodes. In Fig. 2, we show the amount of nodes allocated per job. We observe that the majority of the jobs (around 19 million) uses up to 10 nodes, meaning that the Fugaku workload is mainly composed of jobs using limited resources. However, the dataset contains also around 10K jobs executed on a significant portion of the system, using more than 10K nodes.

Depending on the complexity of the application, HPC jobs can run from a few seconds to several days. In Fig. 3, we show that the dataset covers all values in the range of edge values, with a predominance of the short jobs (around 15 million jobs ran for less than an hour). We observe that jobs running for many days are rare (some hundreds), while thousands of jobs run for around one day.

Being a production system, the jobs submitted to Fugaku are expected to be mainly successful, as users are accounted for their job executions and failed jobs result in additional cost for the computational resources. This can be observed in the distribution of the job *exit state*, in Fig. 4. The figure shows that the great majority of the jobs (more than 21 million) are completed. We can conclude that the dataset is composed of mainly successful jobs, which are fundamental for the correct analysis of job behavior, as no failure alters or stops the execution. Yet, the dataset provides a significant amount of presumably failed jobs (around 2.5 million), which can be used to study the job behavior and investigate the reasons for failed executions.

Figure 5 shows the distribution of the amount of job executions in each month, divided by their *pclass* values (i.e. *memory-bound* or *compute-bound*). We observe a high variability, due to the fact that the system workload and usage changes continuously¹¹. This is witnessed by the fact that the *pclass* distribution throughout the months is neither balanced nor stable; hence, the characteristics of the jobs running on the system are very different. While most of the jobs are *memory-bound* (around 14 million), in some months (e.g. 21/07 and 23/01) the *compute-bound* jobs are the majority.

For job power prediction tasks, the job power consumption is usually normalized per node^{9,32}. In Fig. 6, we show the minimum (*minpcon*), average (*avgpcon*) and maximum (*maxpcon*) job power consumption, normalized on the number of nodes allocated to the job. We observe that the *maxpcon* is shifted to the right (higher values of power consumption) with respect to the other two; the same holds for *avgpcon* with respect to *minpcon*. This is expected, the maximum power consumption is always greater than the average, and the minimum is always the lowest. Again, the dataset covers a wide range of power consumption values, from few watts to more than 200 W.

Limitations. One possible limitation of this dataset is that it does not contain the water usage per job execution, which could be useful to measure the environmental impact of the HPC usage. However, this information is not accessible at the granularity of the nodes, hence it is not possible to link water usage to the single job executions contained in our dataset. We note that the node cooling loop is closed, and the head exchange is conducted to a secondary loop. This secondary loop uses industrial water and evaporation chillers, meaning that the

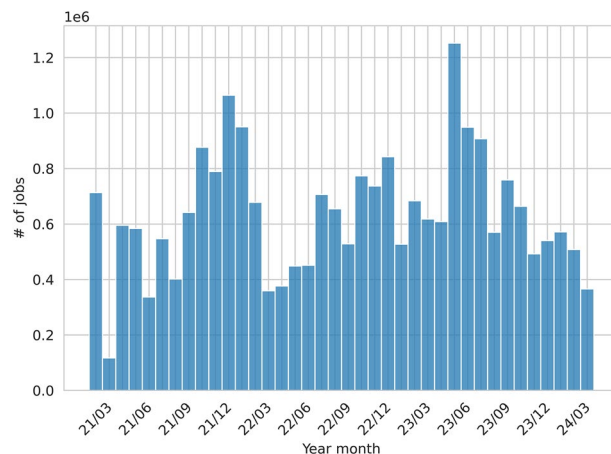


Fig. 1 Distribution of jobs' submission by month.

environmental impact to produce this water is minimal compared to typical household water and the evaporation allows to return the water to nature without contamination.

Moreover, we acknowledge that Fugaku has a unique architecture. However, the A64FX from Fujitsu is an established and commercially available HPC processor, integrated in several HPC clusters. The A64FX is one of the first ARM-based processors with SVE extensions (ARM vector extension) and HBM memory, and the adoption of similar designs is taking over in supercomputing systems and cloud (e.g. NVIDIA Grace, CSCS ALPS and JSC Jupiter, and, AWS graviton family). Despite the specific architecture of the system, the dataset contains real-usage data which only partially depends on the A64FX and Fugaku architecture, as they depend mainly on the users' workload. Furthermore, we note that Fugaku and the other production supercomputers ranked in the Top500²⁰ are used by the national and international scientific and industrial community for state-of-the-art computationally intensive workload, and our dataset is the first and only to release their computational characteristics. The documentation of the Fugaku system and the A64FX are publicly available, and several past work have investigated the Fugaku chip's performance and behavior^{41–44}.

Training ML models on a dataset extracted from a single system can limit the transferability of the models to data extracted from different systems. We note that this depends on the specific hardware/software configuration of the system and the workload executed, rather than on the specific architecture of Fugaku. However, several past work^{45–47} show that it is possible to transfer the knowledge of trained ML models to the data of other systems, by applying fine-tuning or concept-drifting detection techniques. By offering a large and informative dataset, F-DATA serves as a valuable resource for training and testing ML predictive models. The trained models can then be adapted to other systems without the need for extensive data extraction from the target system.

Technical Validation

In this section, we conduct a technical validation of the F-DATA dataset for two purposes: i) showcase the value of the dataset in predictive modeling, and ii) demonstrate that the *sensitive data encoding* improves prediction performance with respect to the anonymized values. As no other dataset contains the workload characteristics and performance of a real Tier0 supercomputer, which F-DATA provides, comparison of our results with those obtained from the other datasets is not entirely feasible and is abstained. In the following sections, we first describe the experimental setup and then present our results.

Experimental Setup. We focus on the prediction of job *exit state*, *pclass*, *avgpcon*, and *duration* values, because they are often addressed in past work. Since the prediction target values are integers (*avgpcon* and *duration*) and labels (*exit state* and *pclass*), we face two regression and two classification tasks.

The experiments are run on a machine equipped with two AMD EPYC 7302 CPUs, 64 cores and 512 GB RAM, running Python 3.11.5 on Linux Fedora 37. The code necessary to repeat the experiments is available at F-DATA GitHub repository⁴⁸.

ML-based predictive models. We need ML-based predictive models that can infer on new unseen data after being trained on historical data, and that are suitable for both regression and classification tasks. We opt for three widely used models, namely XGBoost (XG)⁴⁹, Random Forest (RF)⁵⁰, and *k*-nearest neighbors (KNN)⁵¹. For training, the first two use an ensemble of decision-trees. While XG trains them with a gradient boosting technique, RF does so using different subsets of the data and the features to minimize overfitting. Instead, KNN does not rely on an internal model and therefore does not have an actual training phase. For inference, it computes the *k* most similar elements among the training data, according to a distance metric (e.g. Cosine, Euclidean, Minkowski). The inference is then performed via majority voting on the target values of the *k* elements.

In our experiments, we use the model implementations available in the Python *scikit-learn* library⁵², and instantiate them with the default settings provided by the library.

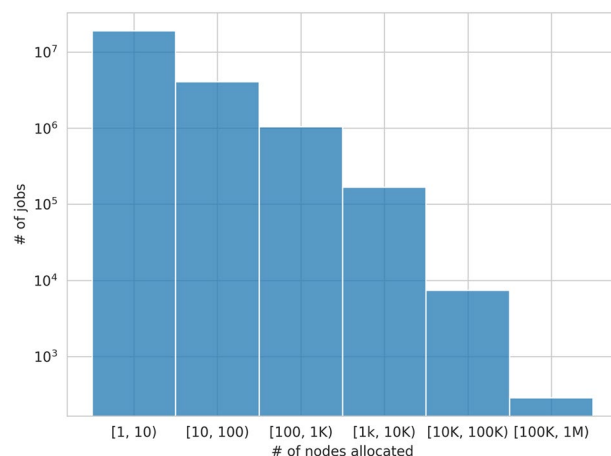


Fig. 2 Distribution of jobs' # of nodes allocated.

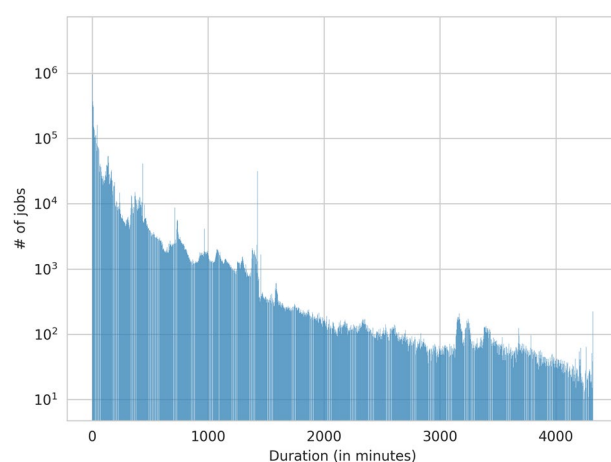


Fig. 3 Distribution of jobs' duration in minutes.

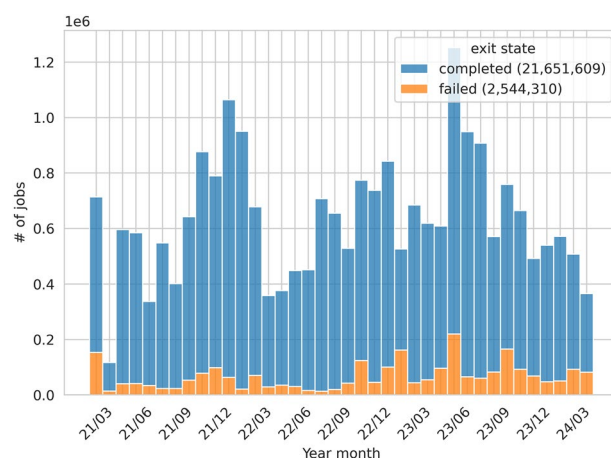


Fig. 4 Amount of jobs executed in each month, divided by *exit state*. In each month, the sum of the two bars (which are stacked on top of each other) gives the total number of jobs submitted in that month.

Evaluation metrics. To evaluate the prediction performance of the models, we adopt simple and widely used metrics. Specifically, for the regression tasks we use the Mean Absolute Error (MAE), and for the classification tasks we use the accuracy. The MAE is computed as the mean of all the absolute error on all the predictions, with respect to the ground truth values, and it is representative of the numerical error of the predictive model. The

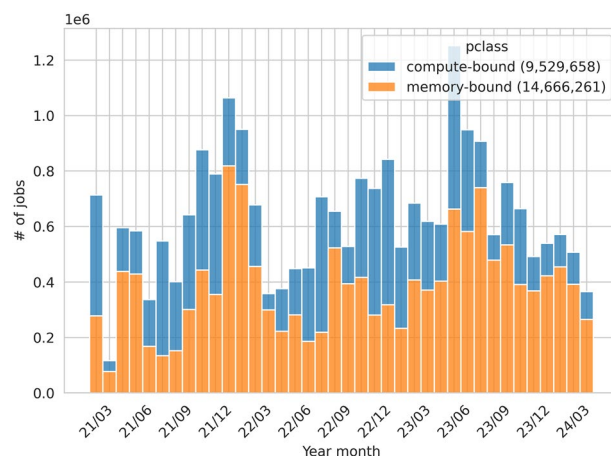


Fig. 5 Amount of jobs executed in each month, divided by *pclass*. In each month, the sum of the two bars (which are stacked on top of each other) gives the total number of jobs submitted in that month.

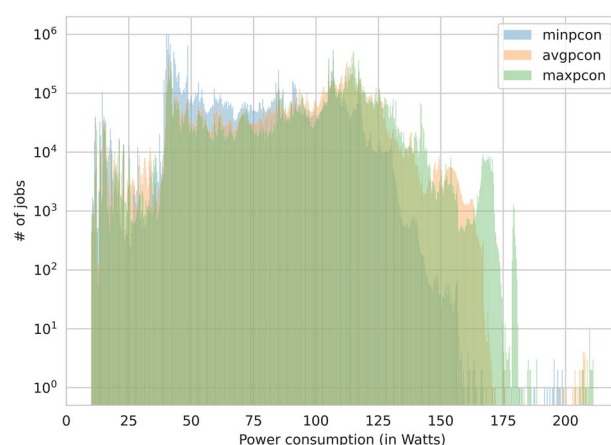


Fig. 6 Distribution of jobs' *minpcn*, *avgpcn* and *maxpcn* per node.

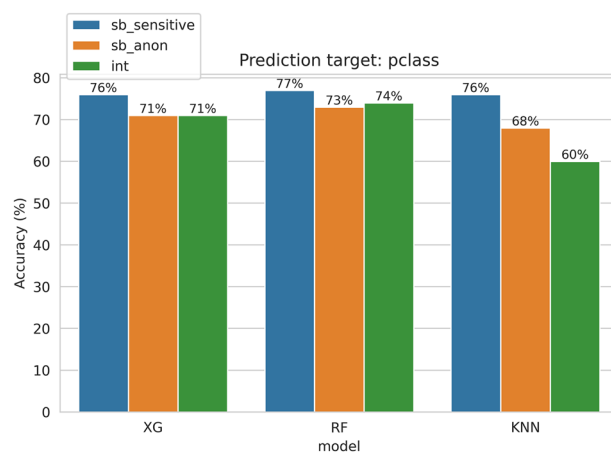


Fig. 7 Model performance on the *pclass* prediction task, divided by the job data encoding. Higher values correspond to better results.

accuracy is a value between 0 and 1, computed as the ratio between the amount of correctly classified values and the size of the whole test set. To ease readability, we multiply it by 100 and express it in percentage form.

Job data preparation. The ML models require the input job data to be encoded in a numerical format, i.e. a list of integers or floating points values. In past work^{9,11}, this was done via an NLP model (SBert) or a standard

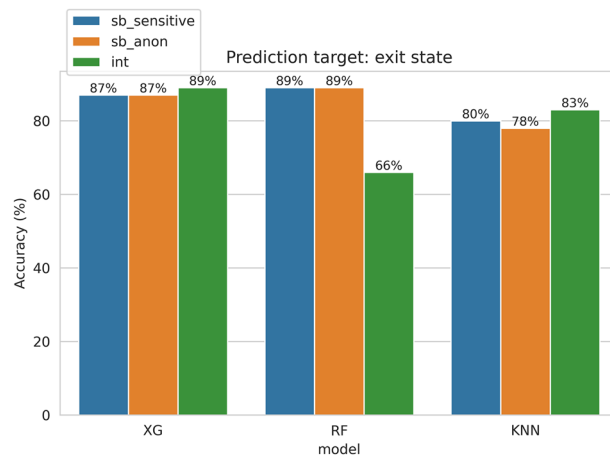


Fig. 8 Model performance on the *exit state* prediction task, divided by the job data encoding. Higher values correspond to better results.

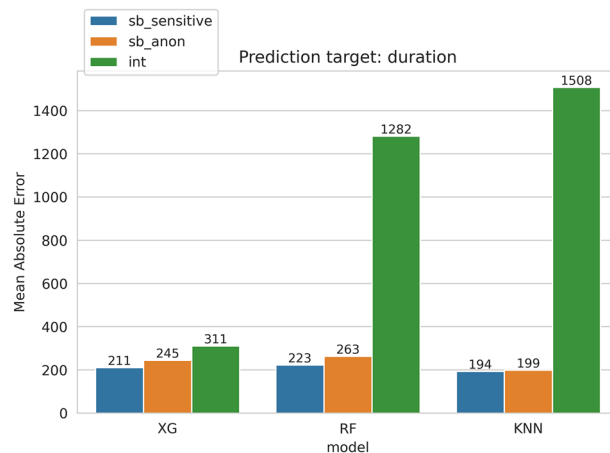


Fig. 9 Model performance on the *duration* prediction task, divided by the job data encoding. Lower values correspond to better results.

integer encoding. In our prediction tasks, we represent each job with its *job name*, *user information* and *job environment*. While these feature values are the most informative about job characteristics, they are also sensitive data and appear in the dataset as anonymized and as *sensitive data encoding* (using Sbert). To adopt the NLP encoding approach^{9,11}, we use i) the *sensitive data encoding* (*sb_sensitive*) as it is, and ii) the Sbert encoding of the comma-separated anonymized data (*sb_anon*) (e.g. “*jobname_0,username_0,jobenvironment_0*”). For the integer encoding (*int*), we assign an integer to all the sensitive feature values, as done previously^{9,11}. We note that this encoding is the same for the original and anonymized values, since each unique value is mapped to an integer with the same order, regardless of the original format.

We train and test all the models using the three different data encoding (*sb_sensitive*, *sb_anon*, and the *int*), for all the prediction tasks. This is done to see whether an NLP-model is able to extract more meaningful information about the job from the original de-anonymized data and compare it to a standard *int* encoding.

Model training and testing. To perform the prediction tasks, it is necessary to define a training set and a testing set. In an HPC context, the data of the testing set needs to always come after, with respect to time, those of the training set, as otherwise the experiments would not be realistic⁹. We consider as the training set the first 26 months of data, namely the jobs executed between 21/03 and 23/05, and we test on the data of the jobs executed between 23/06 and 24/04. The training set is composed of around 19 million job data, while the testing set has the remaining 6 million. We note that this is not an optimal setting for job-centric prediction, as the models are more accurate when they are updated frequently over time with recent data, as researchers have shown^{9–11}. The setting is however sufficient to showcase the utility of the dataset and its features.

Experimental Results. We present our results in Figs. 7–10. We observe that for the *pclass* prediction (Fig. 7), *sb_sensitive* obtains the best results with all three models. In particular, it outperforms *sb_anon*, increasing the accuracy from a minimum of 4% (RF) to a maximum of 8% (KNN). The *int* and the *sb_anon* encodings obtain similar results with XG and RF, while *int* performs the worst with KNN. Concerning the *exit state*

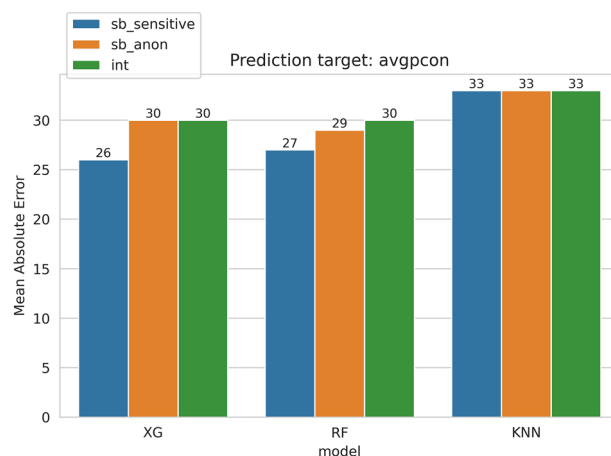


Fig. 10 Model performance on the *avgpcon* prediction task, divided by the job data encoding. Lower values correspond to better results.

prediction ((Fig. 8)), the *sb_sensitive* and *sb_anon* encodings obtain the same results with XG and RF, however, *sb_sensitive* is better with KNN. Here, *int* outperforms the other two with XG and KNN, while it is the worst with RF. These are the only specific cases where *int* yields better results with respect to the *sb_sensitive* and *sb_anon*, which still manage to score very similar results. The usefulness of the *sb_sensitive* is particularly evident in the *duration* ((Fig. 9)) and *avgpcon* prediction tasks (Fig. 10). In both cases, we observe a clear improvement in the prediction performance with respect to the other encodings. The only exception is KNN in *avgpcon*, where all the encodings behave similarly.

This experimental study validates the utility of our dataset for popular predictive modeling tasks, presented in various past work^{10,11,25,32}. Moreover, with the obtained results, we conclude that i) the NLP encoding of the job data usually leads to better prediction than the standard *int* encoding, and ii) the *sensitive data encoding* retains more information about the jobs with respect to anonymized values and further improves the prediction performance, without violating data privacy.

Code availability

The code used to generate the dataset is publicly available in the F-DATA GitHub repository⁴⁸. The repository includes the Python scripts used to clean the data, create the derived features and anonymize the sensitive information. Moreover, we provide the scripts to perform the prediction tasks presented in the previous sections and generate the plots for each dataset. The scripts have been originally executed with *Python3.11* and the library dependencies (listed in the *requirements.txt* file) are as follows:

- matplotlib 3.7.0
- numpy 1.23.5
- numpy 1.22.4
- pandas 1.5.3
- scikit_learn 1.2.1
- seaborn 0.13.2
- sentence_transformers 2.2.2
- tqdm 4.64.0
- xgboost 1.7.4

Received: 27 February 2025; Accepted: 16 July 2025;

Published online: 30 July 2025

References

1. Norman, M. R. *et al.* Unprecedented cloud resolution in a gpu-enabled full-physics atmospheric climate simulation on olcf's summit supercomputer. *The International Journal of High Performance Computing Applications* **36**, 93–105 (2022).
2. Kutzner, C. *et al.* Gromacs in the cloud: A global supercomputer to speed up alchemical drug design. *Journal of Chemical Information and Modeling* **62**, 1691–1711 (2022).
3. Gadioli, D. *et al.* Exscalate: An extreme-scale in-silico virtual screening platform to evaluate 1 trillion compounds in 60 hours on 81 pflops supercomputers. *arXiv preprint arXiv:2110.11644* (2021).
4. Cortés, U. *et al.* The ethical use of high-performance computing and artificial intelligence: fighting covid-19 at barcelona supercomputing center. *AI and Ethics* **2**, 325–340 (2022).
5. Kurth, T. *et al.* Exascale deep learning for climate analytics. In *SC18: International conference for high performance computing, networking, storage and analysis*, 649–660 (IEEE, 2018).
6. Ciesielczyk, T. *et al.* An approach to reduce energy consumption and performance losses on heterogeneous servers using power capping. *Journal of Scheduling* **24**, 489–505 (2021).
7. Borghesi, A., Bartolini, A., Lombardi, M., Milano, M. & Benini, L. Scheduling-based power capping in high performance computing systems. *Sustainable Computing: Informatics and Systems* **19**, 1–13 (2018).
8. Breitbart, J., Pickartz, S., Lankes, S., Weidendorfer, J. & Monti, A. Dynamic co-scheduling driven by main memory bandwidth utilization. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, 400–409 (2017).

9. Antici, F., Yamamoto, K., Domke, J. & Kiziltan, Z. Augmenting ml-based predictive modelling with nlp to forecast a job's power consumption. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Networking, Storage, and Analysis*, 1820–1830 (2023).
10. Menear, K. *et al.* Mastering hpc runtime prediction: From observing patterns to a methodological approach. In *Practice and Experience in Advanced Research Computing*, 75–85 (2023).
11. Antici, F., Borghesi, A. & Kiziltan, Z. Online job failure prediction in an hpc system. In *Euro-Par 2023: Parallel Processing Workshops: Euro-Par 2023 International Workshops, Limassol, Cyprus, August 28–September 1, 2023, Revised Selected Papers* (Springer Nature, 2024).
12. Hameed, A. *et al.* A survey and taxonomy on energy efficient resource allocation techniques for cloud computing systems. *Computing* **98**, 751–774 (2016).
13. Ghiasvand, S. & Ciorba, F. M. Assessing data usefulness for failure analysis in anonymized system logs. In *2018 17th International Symposium on Parallel and Distributed Computing (ISPDC)*, 164–171 (IEEE, 2018).
14. Scargall, S. *Profiling and Performance*, 295–312 (Apress, Berkeley, CA, 2020). https://doi.org/10.1007/978-1-4842-4932-1_15.
15. Hutcheson, A. & Natoli, V. Memory bound vs. compute bound: A quantitative study of cache and memory bandwidth in high performance applications. *Stone Ridge Technology, Internal White Paper* (2011).
16. Antici, F., Seyedkazemi Ardebili, M., Bartolini, A. & Kiziltan, Z. Pm100: A job power consumption dataset of a large-scale production hpc system. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Networking, Storage, and Analysis*, 1812–1819 (2023).
17. Breitbart, J., Weidendorfer, J. & Trinitis, C. Case study on co-scheduling for hpc applications. In *2015 44th International Conference on Parallel Processing Workshops*, 277–285 (2015).
18. Qureshi, B. Profile-based power-aware workflow scheduling framework for energy-efficient data centers. *Future Generation Computer Systems* **94**, 453–467 (2019).
19. Dongarra, J., Luszczek, P. & Heroux, M. Hpcg technical specification. *Sandia National Laboratories, Sandia Report SAND2013-8752* (2013).
20. Dongarra, J. J. *et al.* Top500 supercomputer sites. *Supercomputer* **13**, 89–111 (1997).
21. Antici, F., Bartolini, A., Domke, J., Kiziltan, Z. & Yamamoto, K. F-DATA: A Fugaku Workload Dataset for Job-centric Predictive Modelling in HPC Systems <https://doi.org/10.5281/zenodo.11467483> (2024).
22. Uno, A., Sueyasu, F. & Sekizawa, R. Operations management software of supercomputer fugaku. *Fujitsu Technical Review* 2020–03 (2020).
23. Stonebraker, M. & Rowe, L. A. The design of postgres. *ACM Sigmod Record* **15**, 340–355 (1986).
24. Wahib, M. & Maruyama, N. Scalable kernel fusion for memory-bound gpu applications. In *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 191–202 (IEEE, 2014).
25. Antici, F., Bartolini, A., Kiziltan, Z., Babaoglu, O. & Kodama, Y. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15 (IEEE, 2024).
26. Williams, S. Roofline: An insightful visual performance model for floating-point programs and multicore. *ACM Communications* **16** (2009).
27. Hasan, M. S., de Oliveira, F. A., Ledoux, T. & Pazat, J.-L. Enabling green energy awareness in interactive cloud application. In *2016 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 414–422 (IEEE, 2016).
28. Murthy, S., Bakar, A. A., Rahim, F. A. & Ramli, R. A comparative study of data anonymization techniques. In *2019 IEEE 5th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*, 306–309 (IEEE, 2019).
29. Reimers, N. & Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).
30. Devlin, J. *et al.* BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 NAACL: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186 (Association for Computational Linguistics, Minneapolis, Minnesota, 2019).
31. Xu, Z., Guo, Z. & Cristianini, N. On compositionality in data embedding. In *International Symposium on Intelligent Data Analysis*, 484–496 (Springer, 2023).
32. Borghesi, A., Bartolini, A., Lombardi, M., Milano, M. & Benini, L. Predictive modeling for job power consumption in hpc systems. In *High Performance Computing: 31st International Conference, ISC High Performance 2016, Frankfurt, Germany, June 19–23, 2016, Proceedings*, 181–199 (Springer, 2016).
33. Saillant, T., Weill, J.-C. & Mougeot, M. Predicting job power consumption based on rjms submission data in hpc systems. In *High Performance Computing: 35th International Conference, ISC High Performance 2020, Frankfurt/Main, Germany, June 22–25, 2020, Proceedings* 35, 63–82 (Springer, 2020).
34. Li, B. *et al.* Toward sustainable hpc: Carbon footprint estimation and environmental implications of hpc systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15 (2023).
35. Asifuzzaman, K., Monil, M. A. H., Liu, F. & Vetter, J. S. Evaluating hpc kernels for processing in memory. In *Proceedings of the 2022 International Symposium on Memory Systems*, 1–6 (2022).
36. Orenes-Vera, M., Tureci, E., Wentzlaff, D. & Martonosi, M. Dalorex: A data-local program execution and architecture for memory-bound applications. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 718–730 (IEEE, 2023).
37. Banjongkan, A., Pongsena, W., Kerdprasop, N. & Kerdprasop, K. A study of job failure prediction at job submit-state and job start-state in high-performance computing system: Using decision tree algorithms. *Journal of Advances in Information Technology* **12** (2021).
38. Jassas, M. S. & Mahmoud, Q. H. Analysis of job failure and prediction model for cloud computing using machine learning. *Sensors* **22**, 2035 (2022).
39. Chen, X., Lu, C.-D. & Pattabiraman, K. Predicting job completion times using system logs in supercomputing clusters. In *2013 43rd Annual IEEE/IFIP Conference on Dependable Systems and Networks Workshop (DSN-W)*, 1–8 (IEEE, 2013).
40. Galleguillos, C., Kiziltan, Z., Sirbu, A. & Babaoglu, O. Constraint programming-based job dispatching for modern hpc applications. In *Principles and Practice of Constraint Programming: 25th International Conference, CP 2019, Stamford, CT, USA, September 30–October 4, 2019, Proceedings* 25, 438–455 (Springer, 2019).
41. Matsuoka, S. Fugaku and a64fx: the first exascale supercomputer and its innovative arm cpu. In *2021 Symposium on VLSI Circuits*, 1–3 (IEEE, 2021).
42. Jackson, A., Weiland, M., Brown, N., Turner, A. & Parsons, M. Investigating applications on the a64fx. In *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, 549–558 (IEEE, 2020).
43. Bari, M. A. S. *et al.* A64fx performance: experience on ookami. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, 711–718 (IEEE, 2021).
44. Alappat, C. *et al.* Performance modeling of streaming kernels and sparse matrix-vector multiplication on a64fx. In *2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 1–7 (IEEE, 2020).

45. Madireddy, S. *et al.* Adaptive learning for concept drift in application performance modeling. In *Proceedings of the 48th International Conference on Parallel Processing, ICPP'19* (Association for Computing Machinery, New York, NY, USA, 2019). <https://doi.org/10.1145/3337821.3337922>.
46. Povaliaiev, D., Liem, R., Kunkel, J., Lofstead, J. & Carns, P. High-quality i/o bandwidth prediction with minimal data via transfer learning workflow. In *2024 IEEE 36th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 93–104 (2024).
47. Jagait, R. K., Fekri, M. N., Grolinger, K. & Mir, S. Load forecasting under concept drift: Online ensemble learning with recurrent neural network and arima. *IEEE Access* **9**, 98992–99008 (2021).
48. Antici, F., Bartolini, A., Domke, J., Kiziltan, Z. & Yamamoto, K. F-data: A fugaku workload dataset for job-centric predictive modelling in hpc systems. <https://github.com/francescoantici/F-DATA> (2024). Accessed: 2025-05-11.
49. Chen, T. & Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 785–794 (2016).
50. Breiman, L. Random forests. *Machine learning* **45**, 5–32 (2001).
51. Fix, E. & Hodges, J. L. Discriminatory analysis. nonparametric discrimination: Consistency properties. *International Statistical Review/Revue Internationale de Statistique* **57**, 238–247 (1989).
52. Pedregosa, F. *et al.* Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011).
53. Fujitsu Limited. A64fx pmu events https://raw.githubusercontent.com/fujitsu/A64FX/master/doc/A64FX_PMU_Events_v1.2.pdf (2019).

Acknowledgements

This research was partly funded by the HE EU Graph-Massivizer project (g.a. 101093202), the EuroHPC JU SEANERGY project. This work has been supported by the Spoke “FutureHPC & BigData” and “Multiscale Modelling & Engineering Applications” of the ICSC-Centro Nazionale di Ricerca in “High Performance Computing, Big Data & Quantum Computing”, funded by the EU - NextGenerationEU.

Author contributions

F.A. took care of the creation, publication and technical validation of the dataset. A.B., J.D. and Z.K. helped in the discussion on the dataset structure, as well as on the anonymization technique. K.Y. provided the access to the data, as well as assistance in obtaining the internal validation for the anonymization and disclosure of the dataset. All the authors reviewed the manuscript and contributed in writing.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to F.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025